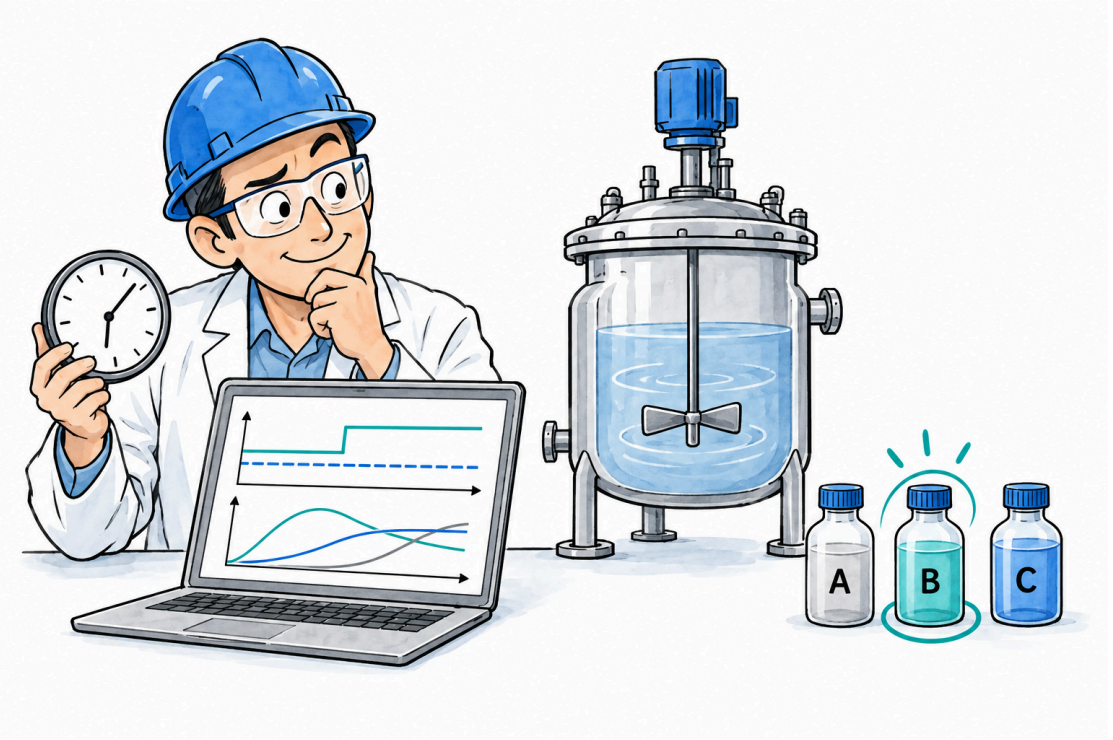
2주차: MLP로 다성분 정상상태 응답 학습하기

연속반응의 전환율과 생성물 조성

A → B → C에서 원하는 중간 생성물은 B입니다. 체류시간이 짧으면 B가 충분히 만들어지지 않고, 길면 더 많은 B가 C로 반응합니다.

전환율 하나로는 제품 조성을 알 수 없으므로 엔지니어는 각 조건의 세 출구 농도를 모두 예측해야 합니다.

한 모델로 A·B·C를 일관되게 예측할 수 있을까요?



성분별 예측 오차

세 농도의 단위는 같아도 변화 폭은 다르므로, 크기를 조정하지 않은 손실함수는 한 성분의 오차를 더 크게 반영할 수 있습니다.

이번 주에는 MLP의 출력 계산, gradient 한 번으로 파라미터가 바뀌는 과정과 성분별 평가를 다뤄봅니다.

정상상태 MLP를 학습하고 확인하기

1. A·B·C의 정상상태 기준값을 유도합니다.
2. 데이터 크기를 조정하고 순전파·역전파·파라미터 갱신을 한 번씩 계산
3. MLP를 학습해 성분별 RMSE와 농도 합 residual을 비교

1. CSTR의 연속반응 A → B → C

기준 공정: 완전혼합·일정 부피·일정 밀도·등온 CSTR의 1차 연속반응 A → B → C. 유입: A 포함, B·C 농도 0. 운전 입력: 온도. 단위: 농도 mol/L, 온도 K, 시간 min

$$\frac{dC_A}{dt} = \frac{C_{Af} - C_A}{\tau} - k_1 C_A$$

$$\frac{dC_B}{dt} = -\frac{C_B}{\tau} + k_1 C_A - k_2 C_B$$

$$\frac{dC_C}{dt} = -\frac{C_C}{\tau} + k_2 C_B$$

속도상수와 운전 영역

두 속도상수의 단위는 1/min이며, 온도에 따라 아래 식으로 계산

$$k_1\left(T\right) = 0.2\exp\left[6000\left(\frac{1}{330} - \frac{1}{T}\right)\right]$$
$$k_2\left(T\right) = 0.1\exp\left[5000\left(\frac{1}{330} - \frac{1}{T}\right)\right]$$

항목	값·범위
기준 온도	330 K
기준 k_1; k_2	0.2; 0.1 /min
온도 민감도 파라미터	6000 K; 5000 K
온도 T	300–360 K
체류시간 τ	1–10 min
유입 A 농도	0.8–1.5 mol/L

세 수지는 각각 A의 소모, B의 생성·소모, C의 생성을 나타냅니다. 세 식을 더하면 반응 항이 상쇄됩니다.

2. 세 출력을 갖는 정상상태 mapping 유도

시간 미분을 0으로 놓고 각 식에 τ를 곱한 뒤 농도 항을 모읍니다.

$$(1 + k_1\tau)\, C_A = C_{\mathrm{Af}}$$
$$(1 + k_2\tau)\, C_B = k_1\tau C_A$$
$$C_C = k_2\tau C_B$$

A·B·C 농도 계산

A를 먼저 구하고 B, C에 차례로 대입합니다.

$$C_A = \frac{C_{\mathrm{Af}}}{1 + k_1\tau}$$

$$C_B = \frac{k_1\tau C_{\mathrm{Af}}}{(1 + k_1\tau)(1 + k_2\tau)}$$

$$C_C = k_2\tau C_B$$

330 K, τ = 5 min, 유입 농도 1.2 mol/L에서 농도는 [0.6, 0.4, 0.2] mol/L. 합은 유입 농도와 같으며, 이 세 농도를 정상상태 MLP의 기준값으로 사용

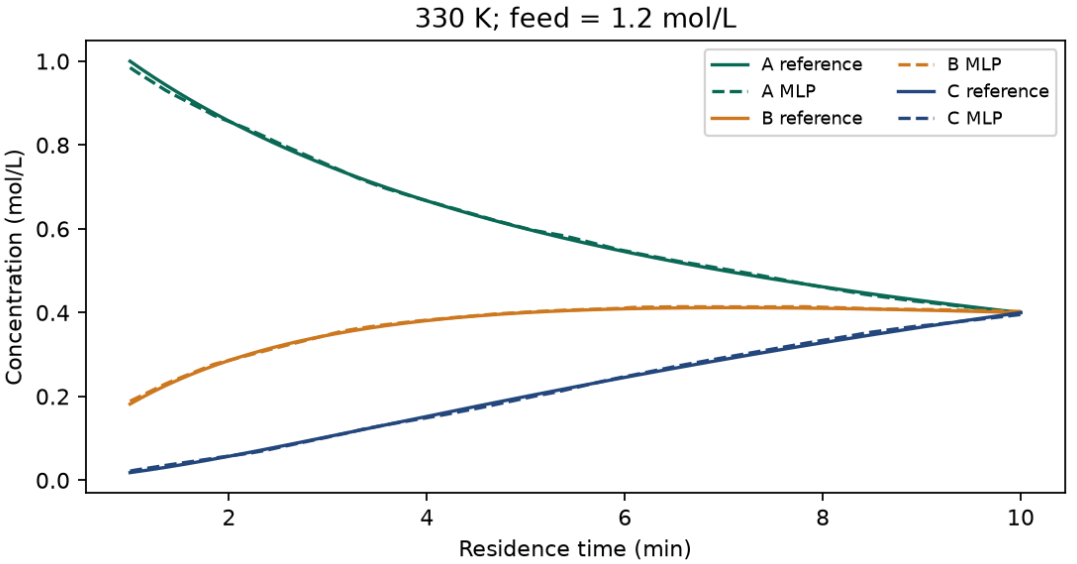
중간 생성물 B의 비단조 응답

온도가 고정되면 C_B 식에서 B의 수율과 그 수율을 최대화하는 체류시간을 구할 수 있습니다.

$$Y_B = \frac{C_B}{C_{\mathrm{Af}}} = \frac{k_1\tau}{\left(1+k_1\tau\right)\left(1+k_2\tau\right)}, \quad \tau^* = \frac{1}{\sqrt{k_1 k_2}}$$

세 성분의 농도 profile 비교

330 K에서 τ*는 약 7.07 min. 짧은 체류시간에서는 B 생성이 부족하고, 체류시간이 길어지면 더 많은 B가 C로 반응합니다.



330 K에서의 다성분 응답. B 농도는 중간 체류시간에서 최대가 된다.

3. 학습 데이터 구성

한 행의 입력은 [T, τ, C_Af], 출력은 [C_A, C_B, C_C]이며 두 배열의 크기는 모두 (N, 3). 표의 운전 영역에서 train 400개, validation 120개, test 160개를 생성하고 정상상태 식으로 기준값을 계산

Train 데이터에서 세 농도의 표준편차는 약 [0.283, 0.110, 0.217] mol/L로, 단위가 같아도 변화 폭은 다릅니다.

4. Forward 계산과 scaling

MLP 구조: 3 → 32 → 32 → 3. ReLU 은닉층과 선형 출력층

은닉층 계산: 가중합 + bias → ReLU(a) = max(0, a). 양수는 통과, 음수는 0. 농도 데이터로 학습하는 파라미터: 가중치와 bias

$$h_1 = \mathrm{ReLU}\left(W_1 \tilde{\mathrm{x}} + b_1\right)$$
$$h_2 = \mathrm{ReLU}\left(W_2 h_1 + b_2\right)$$
$$\hat{z} = W_3 h_2 + b_3$$

층	Weight shape	Bias shape	Parameter 수
입력 → hidden 1	32 × 3	32	128
Hidden 1 → hidden 2	32 × 32	32	1056
Hidden 2 → 출력	3 × 32	3	99
합계			**1283**

Scaling과 학습 loss

입력은 명시한 하한·상한으로 크기를 조정합니다. 출력은 train 평균 μ와 표준편차 s로 표준화하고, 예측한 뒤 원래 단위로 복원

$$\tilde{x}_j = 2\frac{x_j - l_j}{u_j - l_j} - 1$$

$$z_j = \frac{y_j - \mu_j}{s_j}, \quad \hat{y}_j = \mu_j + s_j \hat{z}_j$$

손실함수는 표본과 표준화된 출력 성분 전체의 제곱 오차를 평균합니다.

$$L = \frac{1}{3N} \sum_{n=1}^{N} \sum_{j=1}^{3} \left(\hat{z}_{n,j} - z_{n,j} \right)^2$$

각 농도 오차를 해당 성분의 train 변화 폭에 상대적으로 평가하므로, 표준편차가 작은 B의 원래 단위 제곱 오차에는 더 큰 가중치가 붙습니다.

5. Backpropagation 한 번 계산하기

계산 예제: bias가 모두 0인 3 → 2 → 3 신경망. 손실함수: $L = \frac{1}{2}\|\hat{y} - y\|^2$. 학습률: 0.1

$$x = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}, y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

$$W_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, W_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

은닉층의 활성화 전 값 a와 활성화 값 h, 예측값과 오차를 구합니다.

$$a = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, h = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \hat{y} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, e = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$$

Gradient와 파라미터 갱신

출력 오차를 출력층 가중치와 ReLU 미분을 거쳐 전달합니다.

$$\delta_2 = e, \nabla_{W_2} L = eh^T = \begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 1 & 0 \end{bmatrix}$$

$$\delta_1 = \left({W_2}^T e \right) \odot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

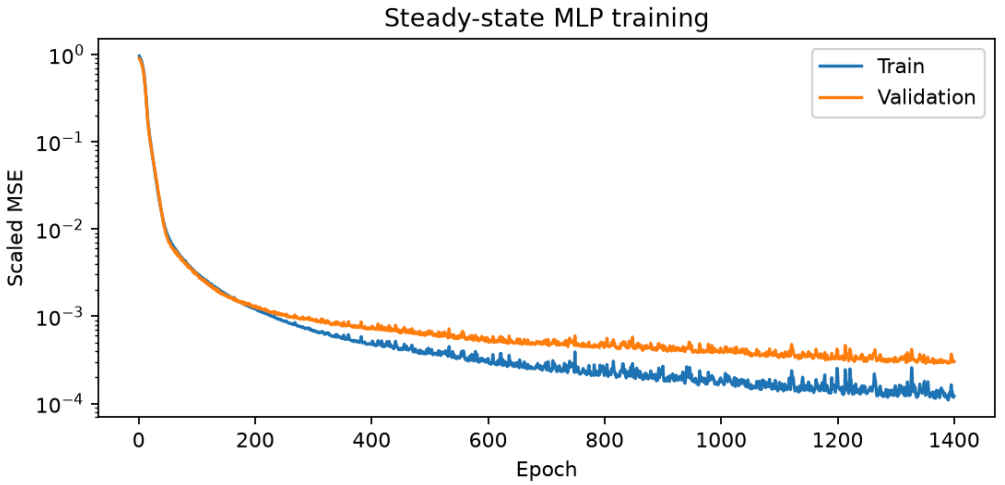
$$\nabla_{W_1} L = \delta_1 x^T = \begin{bmatrix} 2 & 0 & -2 \\ 0 & 0 & 0 \end{bmatrix}$$

Bias gradient는 각각 δ_1, δ_2입니다. 모든 가중치와 bias를 parameter ← parameter − 0.1 × gradient로 갱신하면 다음 순전파 출력은 [0.26, 0.14, 0.26]이고, loss는 1.5에서 0.4374로 줄어듭니다. Notebook의 autograd 결과와 손계산을 대조합니다.

6. 정상상태 모델 학습과 평가

학습 설정: Adam, 학습률 0.001, batch 64, 최대 1400 epoch. 가중치 선택: 매 epoch의 validation loss 중 최솟값

```
prediction = model(inputs)
loss = loss_function(prediction, targets)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

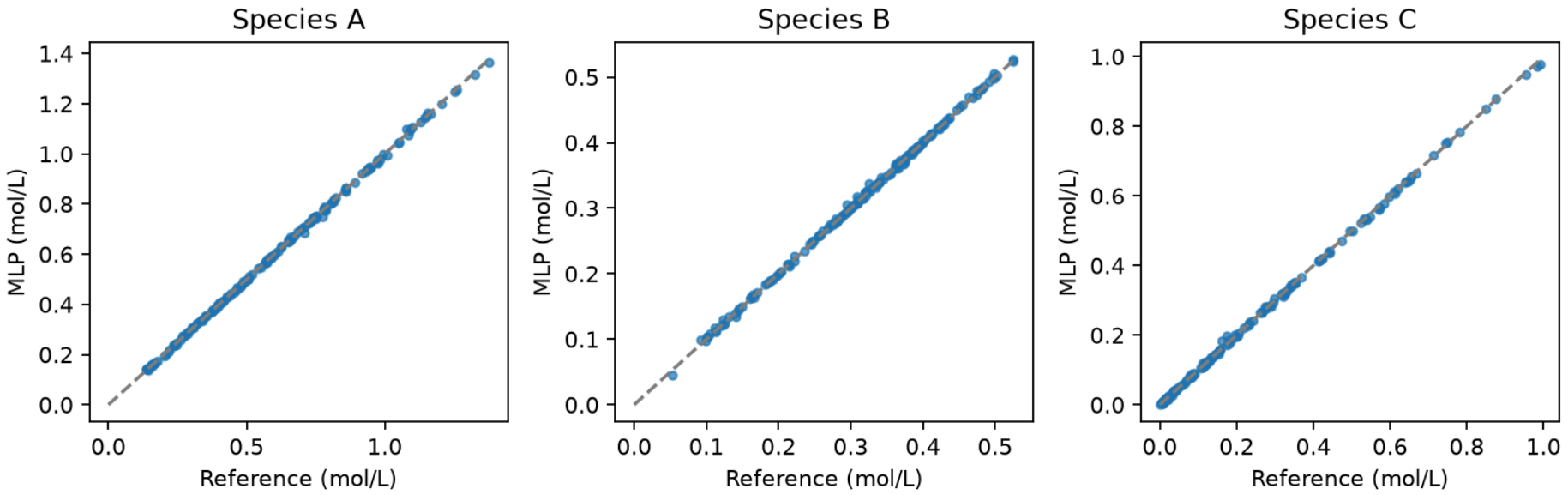


Train·validation loss. 아래 예측 지표는 mol/L 단위로 되돌린 뒤 계산한다.

출력 성분별 평가

Test RMSE, mol/L	A	B	C
출력 scaling 적용	0.005275	0.002940	0.004423
출력 scaling 미적용	0.006634	0.003917	0.004162

이번 실행에서는 출력 scaling으로 A·B RMSE가 줄었고 C RMSE는 조금 커졌습니다. 각 성분의 변화를 따로 확인해야 전체 오차 하나에 가려진 차이를 볼 수 있습니다.



성분별 parity plot에서 각 출력의 오차를 확인한다.

농도 합 확인

$$r = \hat{C}_A + \hat{C}_B + \hat{C}_C - C_{\mathrm{Af}}$$

Test residual RMSE는 0.003781 mol/L. 성분별 오차로 예측 정확도를, 농도 합 residual로 해당 수지의 일관성을 평가

수지를 만족하는 기준값으로 학습해도 세 선형 출력의 합이 유입 농도와 같도록 강제되지는 않습니다.

유입 1.2 mol/L에서 예측이 [0.62, 0.41, 0.20] mol/L이면 합은 1.23 mol/L이고 r = 0.03 mol/L입니다.
몰분율을 예측할 때도 합이 1을 벗어날 수 있습니다.

4주차 예고: 농도 합 제약의 모델 반영

학습 후 출력을 보정하거나, 제약을 만족하는 변환을 학습 과정에 포함할 수 있습니다.

접근	보정이 들어가는 위치
학습 후 외부 mapping/projection	Raw predictor를 학습한 뒤 출력을 보정한다.
KKT-hPINN	입력과 raw 예측을 받는 고정 affine projection을 loss 전에 적용하며, 예측할 때도 적용한다.

$$x \rightarrow f_\theta \rightarrow \hat{y} \rightarrow \Pi_x \rightarrow \tilde{y} \rightarrow \text{loss}$$

KKT-hPINN은 투영한 출력으로 loss를 계산하고 투영층을 거쳐 역전파합니다. 투영 계수는 고정하지만 MLP 가중치는 이 변환을 반영해 학습합니다.

투영층은 지정한 선형 등식을 수치 연산 오차 범위에서 만족시키며, 비음수성에는 별도 제약이 필요합니다.

출처: Chen et al. (2024), DOI: 10.1016/j.compchemeng.2024.108764.

7. 필수 정상상태 실습 실행

정상상태 Notebook을 받습니다.

```
python -m pip install -r requirements.txt
python week02_lab.py --mode steady --output-dir week02-results
```

학습한 예측 모델의 성분별 RMSE와 농도 합 residual을 평가

8. 필수 연습문제와 제출물

1. C_A·C_B·C_C를 유도해 합을 확인하고, B 곡선이 단조롭지 않은 이유를 설명합니다.
2. 정상상태 MLP의 파라미터 1283개와 손으로 계산한 gradient 갱신을 확인
3. 출력 scaling 전후의 성분별 RMSE를 비교
4. 독립 입력에서 농도 합 residual과 최소 예측 농도를 살펴봅니다.

제출물: 정상상태 모델, 데이터 분할·scaling 정보, 확인한 gradient 갱신, 성분별 오차·일관성 평가

구현 설명은 PyTorch의 모델 구성, autograd, 학습을 참고합니다.

읽기자료

2주차 읽기자료 · 국문 PDF

KKT-hPINN 참고문헌: Chen, H., Constante Flores, G. E., & Li, C. (2024). Physics-informed neural networks with hard linear equality constraints. *Computers & Chemical Engineering*, 189, 108764.